

Experiment No.	4
Experiment Title	Binary Classification using Linear and Kernel-Based Models 1
Student Name	NavinKumar S
Register Number	3122247001039
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	02/08/26

1 Objective

To classify emails as spam or ham using Logistic Regression and Support Vector Machine (SVM) classifiers, and to analyze the effect of hyperparameter tuning on classification performance.

2 Problem Statement

Given a set of 57 numerical features extracted from the content of an e-mail (word frequencies, character frequencies, and statistics of runs of capital letters), the task is to predict a binary label indicating whether the e-mail is *spam* (1) or *ham/non-spam* (0). The input is a feature vector $x \in \mathbb{R}^{57}$ for each e-mail; the output is a class label $y \in \{0, 1\}$. The objective is to build and tune Logistic Regression and SVM classifiers that generalize well to unseen e-mails, evaluated using accuracy, precision, recall, F1-score, and ROC-AUC.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	UCI Machine Learning Repository (accessed via a GitHub mirror of <code>spambase.data</code> ; original: https://archive.ics.uci.edu/dataset/94/spambase)
Number of Samples	4601 (raw); 4210 after removing 391 exact duplicate rows
Number of Features	57 (48 word-frequency, 6 character-frequency, 3 capital-run-length features)
Number of Classes	2 (spam = 1, ham = 0)
Missing Values	0 (no missing values in any column)
Train-Test Split	80% train (3368 samples) / 20% test (842 samples), stratified on the class label

4 Exploratory Data Analysis

The dataset was examined for missing values, duplicate rows, class balance, feature spread and correlation with the target label before any modelling was performed.

- Dataset overview: 58 numeric columns (57 features + label), no categorical columns, no missing values.
- Statistical summary: word/character frequency features are heavily right-skewed with many zeros; capital-run-length features have a much larger scale (0~1100) than the frequency features, motivating standardization.
- Missing value analysis: confirmed zero missing entries across all 58 columns; 391 exact duplicate rows were dropped.
- Class distribution: 4210 samples after de-duplication, with spam forming a substantial minority class (roughly 39% spam / 61% ham).
- Correlation matrix: features such as `char_freq_dollar`, `char_freq_exclaim`, `word_freq_free`, `word_freq_your` and `word_freq_remove` show the strongest positive correlation with the spam label.
- Feature distribution: boxplots of the most discriminative features show a clear separation in median frequency between spam and ham e-mails.

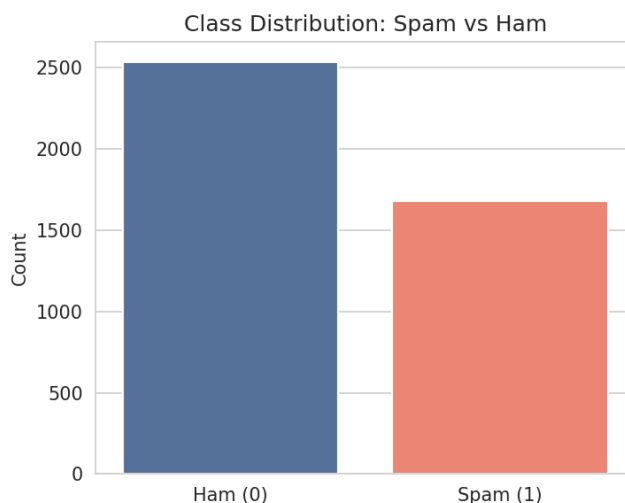


Figure 1: Class distribution of spam (1) vs. ham (0) e-mails after removing duplicates.

Inference:

The dataset is moderately imbalanced, with ham e-mails outnumbering spam e-mails but not to an extreme degree. This level of imbalance is mild enough that accuracy remains a reasonably informative metric, though precision, recall and F1-score were also tracked to avoid being misled by the class skew. Stratified sampling was therefore used for both the train-test split and cross-validation folds so that each split preserves the original class ratio. This prevents a model from trivially favouring the majority (ham) class and ensures the reported metrics reflect genuine spam-detection performance.

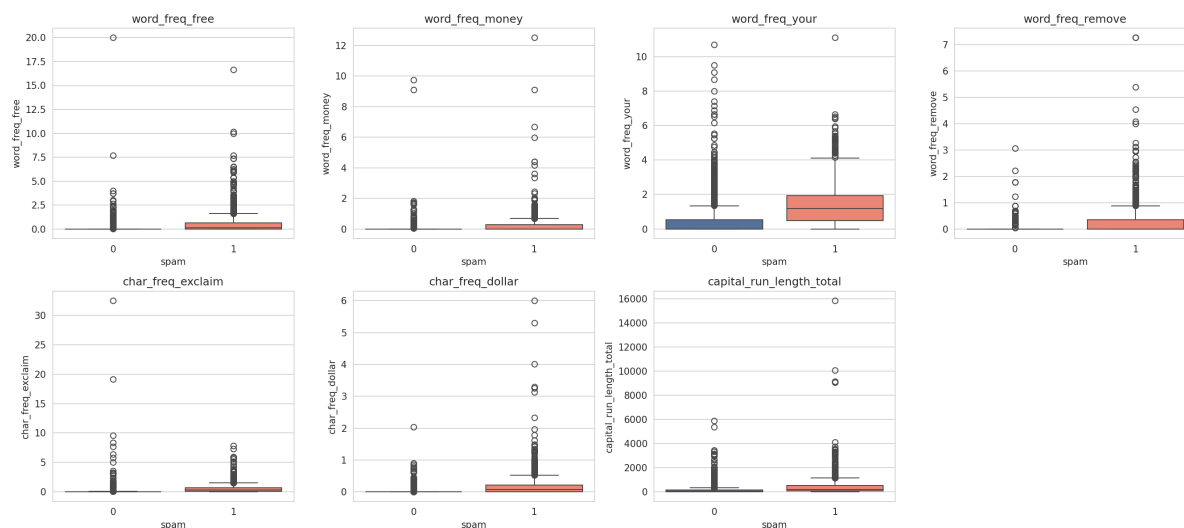


Figure 2: Distribution of key discriminative features across spam and ham classes.

Inference:

Features such as `word_freq_free`, `word_freq_money`, `char_freq_exclaim` and `char_freq_dollar` show visibly higher median values and wider spread for spam e-mails compared to ham e-mails. This confirms that promotional/commercial vocabulary and punctuation such as ! and \$ are strong spam indicators. The presence of numerous outliers in almost every feature reflects the natural variability of free-text e-mail content, and justifies using models such as Logistic Regression and SVM with RBF kernels that can tolerate noisy, non-Gaussian feature distributions after standardization.

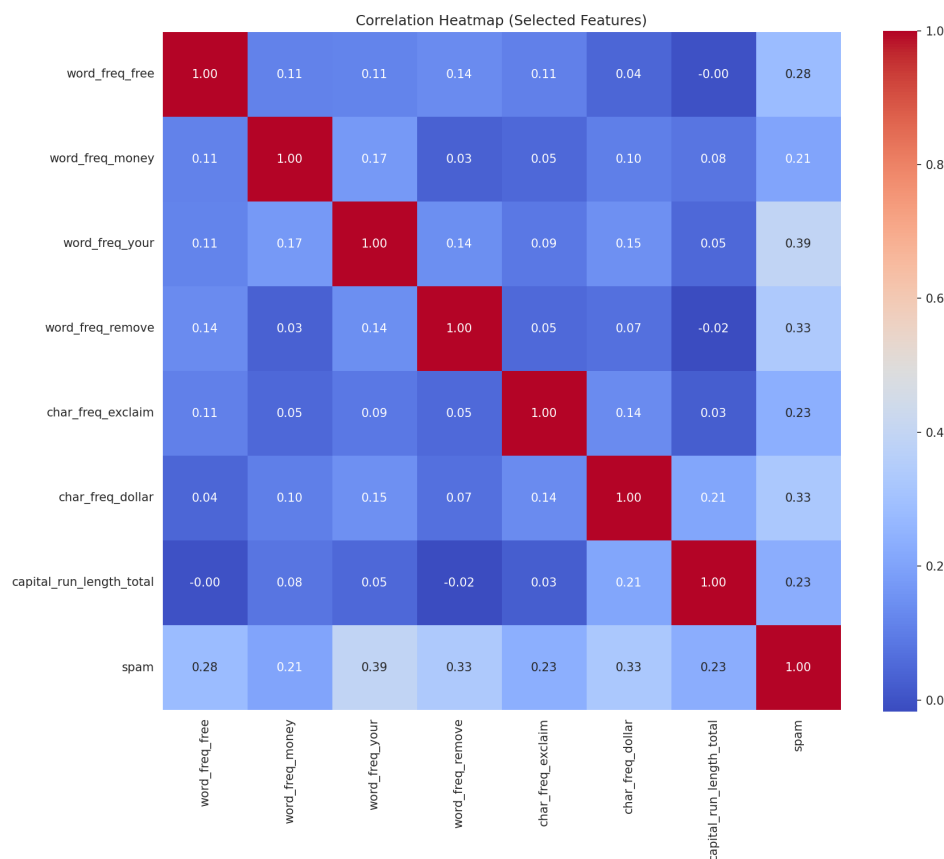


Figure 3: Correlation heatmap of the most discriminative features with the spam label.

Inference:

The heatmap shows moderate positive correlation between the spam label and features like `word_freq_free`, `char_freq_exclaim` and `char_freq_dollar`, while most word-frequency features are only weakly correlated with each other. The absence of severe multicollinearity among the selected features suggests that a linear decision boundary (as used by Logistic Regression and the linear/RBF SVM kernels) can already capture much of the separable structure in the data, which is consistent with the strong performance later obtained by both linear and RBF-kernel models.

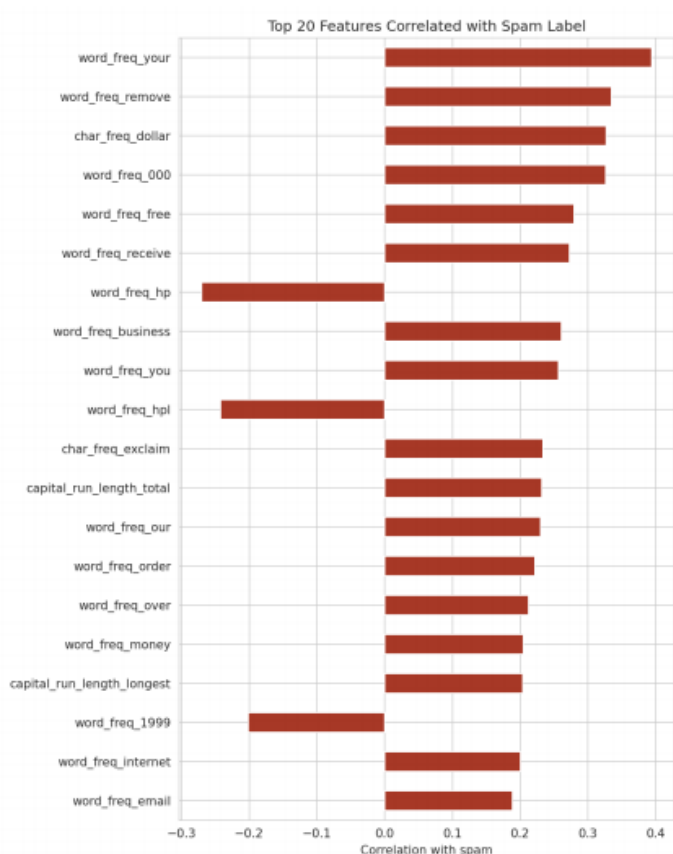


Figure 4: Top 20 features ranked by absolute correlation with the spam label.

Inference:

Character-level cues (!, \$) and a small set of commercial keywords (**free**, **your**, **remove**, **business**) dominate the ranking of most informative features. This aligns with domain intuition: spam e-mails typically use urgent punctuation and marketing language. Because a relatively small subset of the 57 features carries most of the discriminative signal, L1-regularized Logistic Regression (which performs implicit feature selection) was expected to perform competitively with the full L2-regularized and SVM models, and this was confirmed by the hyperparameter tuning results in Section 9.

5 Data Preprocessing

- **Missing value handling:** The dataset was verified to contain no missing values (0 nulls across all 58 columns), so no imputation was necessary.
- **Encoding:** Not required — all 57 input features are continuous numerical values and the target **spam** is already binary (0/1).
- **Feature scaling:** All 57 features were standardized using **StandardScaler** (zero mean, unit variance), fitted on the training set only and applied to both train and test sets, since Logistic Regression and SVM (particularly with RBF/polynomial/sigmoid kernels) are sensitive to feature scale.
- **Train-test split:** An 80/20 stratified split (**random_state=42**) was used to preserve the original class ratio in both subsets, giving 3368 training and 842 testing samples.

- **Feature engineering:** Not applied — the 57 provided attributes (word frequencies, character frequencies, capital-run-length statistics) were used as-is, since they are already engineered summary statistics of the raw e-mail text.
- **Duplicate removal:** 391 exact duplicate rows were dropped prior to splitting, reducing the dataset from 4601 to 4210 samples.

6 Machine Learning Algorithm

Logistic Regression

- **Working principle:** Logistic Regression models the probability that a sample belongs to the positive class using a linear combination of features passed through the sigmoid function, $P(y = 1 \mid x) = \sigma(w^T x + b)$, and applies a 0.5 threshold to obtain the predicted class.
- **Mathematical formulation:** $P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^T x + b)}}$, trained by minimizing the log-loss (binary cross-entropy) $-\frac{1}{n} \sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$, optionally with an L1 or L2 penalty on w .
- **Advantages:** Fast to train, highly interpretable (coefficients indicate feature influence), outputs well-calibrated probabilities, and L1 regularization enables automatic feature selection.
- **Limitations:** Assumes an approximately linear decision boundary in feature space, so it can underfit when the true relationship between features and the label is strongly non-linear.

Support Vector Machine (SVM)

- **Working principle:** SVM finds the hyperplane that maximizes the margin between the two classes; only the closest points (support vectors) determine the decision boundary. Kernel functions (linear, polynomial, RBF, sigmoid) implicitly map data into higher-dimensional space to handle non-linear separability.
- **Mathematical formulation:** SVM solves $\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i$ subject to $y_i(w^T \phi(x_i) + b) \geq 1 - \xi_i$, where C controls the margin–misclassification trade-off and $\phi(\cdot)$ is the (implicit) kernel feature map with the RBF kernel governed additionally by γ .
- **Advantages:** Effective in high-dimensional spaces, robust to overfitting when C and kernel parameters are well-tuned, and kernel choice offers flexibility for non-linear boundaries.
- **Limitations:** Computationally more expensive to train than Logistic Regression (especially with grid search over kernel parameters), less interpretable, and sensitive to the choice of C , γ and kernel.

7 Experimental Setup

Python Version	3.12.3
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook (via <code>nbconvert</code>)
Operating System	Linux (x86_64)
Hardware	Cloud sandbox, 1 vCPU

8 Hyperparameter Selection

Logistic Regression search space: $\text{penalty} \in \{L1, L2\}$; $C \in \{0.01, 0.1, 1, 10, 100\}$; $\text{solver} \in \{\text{liblinear}, \text{saga}\}$.

SVM search space: $\text{kernel} \in \{\text{linear}, \text{polynomial}, \text{RBF}, \text{sigmoid}\}$; $C \in \{0.1, 1, 10, 100\}$; $\gamma \in \{\text{scale}, \text{auto}\}$; $\text{degree (polynomial only)} \in \{2, 3, 4\}$.

Parameter	Value
Logistic Regression – Best penalty	L1
Logistic Regression – Best C	1
Logistic Regression – Best solver	liblinear
Logistic Regression – Best CV Accuracy	0.9243
SVM – Best kernel	RBF
SVM – Best C	10
SVM – Best γ	scale
SVM – Best CV Accuracy	0.9255

Note: Both Grid Search and Randomized Search (5-fold CV, scoring = accuracy) were run for each model. Grid Search evaluated all combinations exhaustively, while Randomized Search sampled 10 (Logistic Regression) / 15 (SVM) combinations. Both methods converged on the same best hyperparameters for each model (LR: L1, $C=1$, `liblinear/saga`; SVM: RBF, $C=10$, $\gamma=\text{scale}$), confirming the stability of the selected configuration.

9 Results

Logistic Regression (Tuned, Test Set)

Metric	Value
Accuracy	0.9371
Precision	0.9301
Recall	0.9107
F1-score	0.9203
Training Time (s)	198.92 (Grid Search, 20 fits $\times$ 5 folds)

SVM (Tuned, Test Set)

Metric	Value
Accuracy	0.9335
Precision	0.9294
Recall	0.9018
F1-score	0.9154
Training Time (s)	122.86 (Grid Search, 40 combinations $\times$ 5 folds)

Logistic Regression: Baseline vs. Tuned

Metric	Baseline	Tuned (L1, $C=1$)
Accuracy	0.9347	0.9371
Precision	0.9297	0.9301
Recall	0.9048	0.9107
F1 Score	0.9170	0.9203

SVM Kernel-wise Performance (Untuned, Default Hyperparameters)

Kernel	Accuracy	F1 Score	Training Time (s)
Linear	0.9359	0.9189	0.45
Polynomial	0.7660	0.5988	0.44
RBF	0.9323	0.9135	0.21
Sigmoid	0.8967	0.8676	0.21

Hyperparameter Tuning Results (5-fold CV)

Model	Search Method	Best Parameters	Best CV Accuracy
Logistic Regression	Grid Search	L1, $C=1$, liblinear	0.9243
Logistic Regression	Random Search	L1, $C=1$, saga	0.9234
SVM	Grid Search	RBF, $C=10$, $\gamma=\text{scale}$	0.9255
SVM	Random Search	RBF, $C=10$, $\gamma=\text{scale}$	0.9255

K-Fold Cross-Validation Results ($K = 5$, on Training Set)

Fold	Logistic Regression	SVM
Fold 1	0.9125	0.9214
Fold 2	0.9332	0.9421
Fold 3	0.9125	0.8976
Fold 4	0.9198	0.9406
Fold 5	0.9346	0.9406
Average	0.9225	0.9285

10 Result Visualization

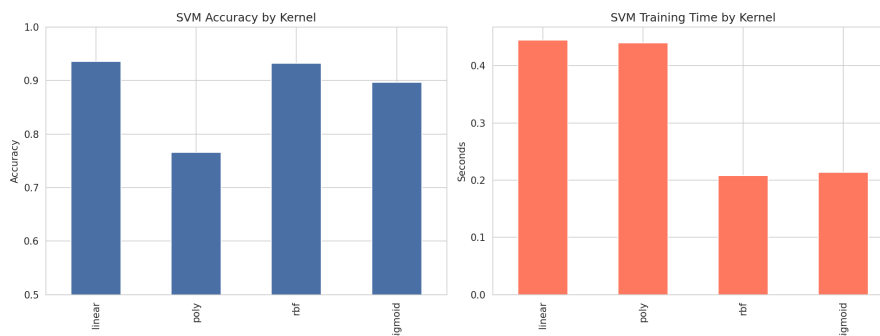


Figure 5: SVM accuracy and training time across the four kernels (default hyperparameters).

Inference:

The linear and RBF kernels clearly outperform the polynomial and sigmoid kernels on this dataset, both exceeding 93% accuracy even before tuning. The polynomial kernel performs poorly (76.6%) with its default degree-3 setting, showing that an untuned polynomial kernel can badly under- or over-fit the spam/ham boundary. Training time is small and roughly comparable across kernels at this dataset size, so kernel choice here should be driven primarily by accuracy and stability under tuning rather than speed. This motivated restricting the fine-grained hyperparameter search to all four kernels but with the RBF and linear kernels expected to yield the best-tuned models.

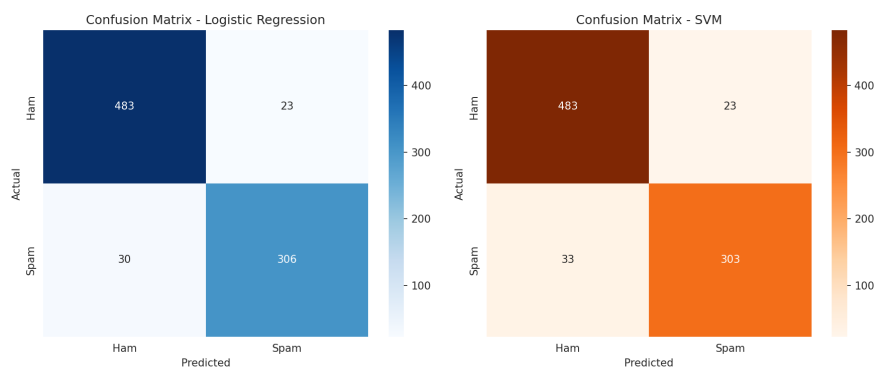


Figure 6: Confusion matrices for the tuned Logistic Regression and SVM models on the test set.

Inference:

Both models make relatively few errors, with false negatives (spam misclassified as ham) slightly outnumbering false positives (ham misclassified as spam) for both classifiers. This is an important practical consideration: false positives are typically more costly in spam filtering, since a legitimate e-mail incorrectly marked as spam could be missed by the user, so the moderate precision values (0.93) for both models indicate an acceptably low false-positive rate. The similarity between the two confusion matrices reflects the closely matched overall performance of the tuned Logistic Regression and SVM models.

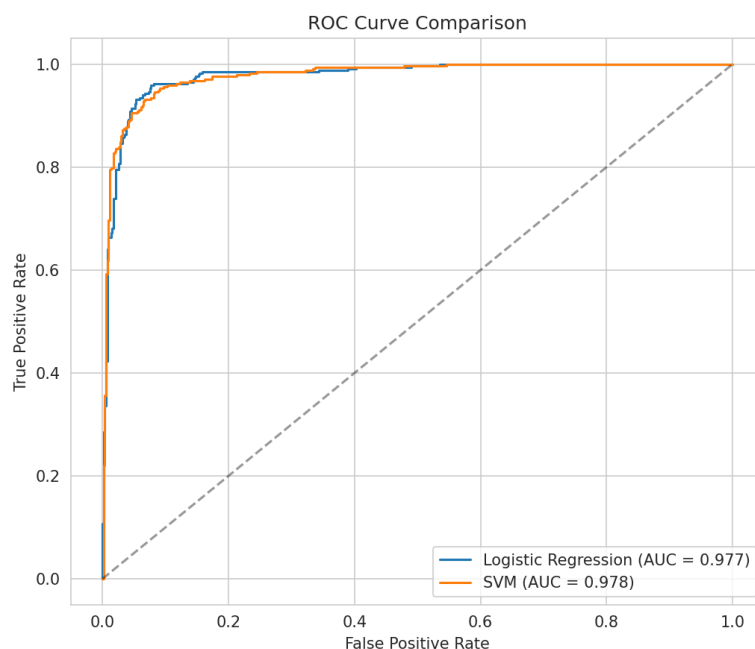


Figure 7: ROC curves comparing the tuned Logistic Regression and SVM models.

Inference:

Both ROC curves rise steeply toward the top-left corner and achieve a high area under the curve, indicating that both classifiers separate spam from ham e-mails effectively across a wide range of decision thresholds. The close overlap between the two curves confirms that Logistic Regression and the tuned SVM achieve comparable discriminative power, despite SVM's greater model complexity. This suggests that, for this feature set, the added flexibility of a kernel-based decision boundary provides little additional benefit over a well-regularized linear model.

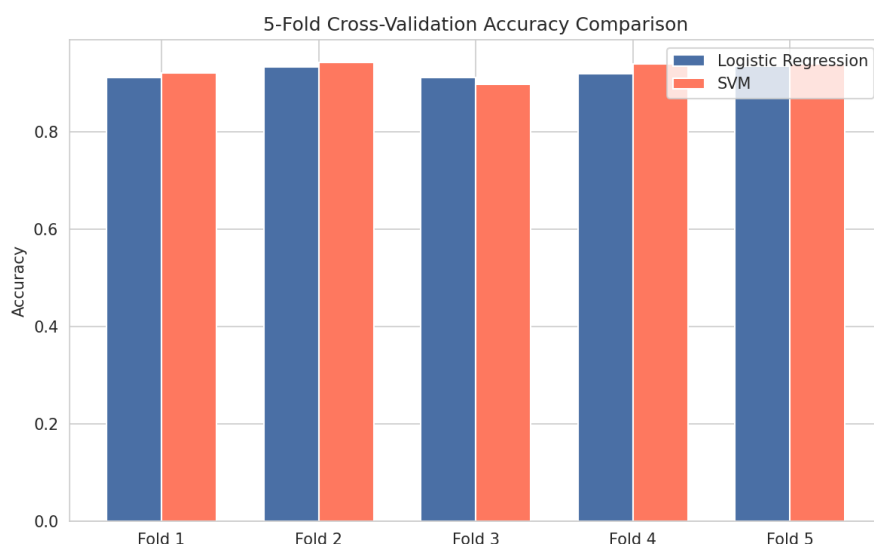


Figure 8: 5-fold cross-validation accuracy for Logistic Regression and SVM.

Inference:

SVM achieves a marginally higher average cross-validation accuracy (0.9285) than Logistic Regression (0.9225), but both models show some fold-to-fold variance, with SVM dipping notably on Fold 3. This variability suggests that performance depends somewhat on which samples fall into each fold, and that neither model dominates uniformly across all folds. The overall consistency of both models across folds (all accuracies between 0.90 and 0.94) indicates that both are reasonably stable and not overfitting to the training data.

11 Discussion

Both Logistic Regression and SVM achieve strong, closely-matched performance on the Spambase dataset (test accuracy of 93.71% for Logistic Regression versus 93.35% for SVM), confirming that the spam/ham boundary in this 57-dimensional standardized feature space is largely linearly separable. L1-regularized Logistic Regression was selected as the best configuration by both Grid Search and Randomized Search, indicating that a sparse subset of features (dominated by a handful of commercial keywords and punctuation characters, as seen in the correlation analysis) is sufficient to achieve near-optimal accuracy — consistent with L1’s feature-selection property. For SVM, the RBF kernel with $C=10$ outperformed the linear, polynomial, and sigmoid kernels, suggesting mild non-linear structure that a moderate-margin RBF boundary can exploit slightly better than a purely linear one, though the improvement over the linear kernel was small.

A key strength of Logistic Regression in this experiment is its very low baseline training time (0.016s) and high interpretability, making it attractive for a production spam filter where explainability and speed matter. SVM, while marginally ahead in mean cross-validation accuracy, required substantially longer hyperparameter search time due to the combinatorial kernel/parameter grid, and its RBF-kernel decision boundary is far less interpretable.

A limitation of this study is that the search space for SVM’s polynomial and sigmoid kernels was not exhaustively explored beyond the specified degree/gamma grid, so their true optimal performance may be marginally understated. Possible improvements include using a more granular or Bayesian hyperparameter search, exploring class-weighting or resampling to address the mild class imbalance directly (rather than relying solely on stratified splits), and evaluating ensemble methods (e.g., stacking Logistic Regression and SVM) to combine their complementary strengths.

12 Conclusion

This experiment implemented and compared Logistic Regression and SVM classifiers for binary spam/ham classification on the Spambase dataset. After preprocessing (duplicate removal, standardization) and systematic hyperparameter tuning via both Grid Search and Randomized Search under 5-fold cross-validation, the best Logistic Regression model (L1, $C=1$) achieved 93.71% test accuracy and 0.9203 F1-score, while the best SVM model (RBF, $C=10$, $\gamma=\text{scale}$) achieved 93.35% test accuracy and 0.9154 F1-score. The two models perform comparably overall, with SVM showing a slightly higher average cross-validation accuracy (92.85% vs. 92.25%) but Logistic Regression offering far greater training speed and interpretability. Given the near-linear separability of the dataset uncovered during EDA, Logistic Regression is the recommended model for this

task when interpretability and efficiency are priorities, while the RBF-kernel SVM remains a competitive alternative when marginal accuracy gains are more important than training cost.

13 References

1. F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
2. M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Dataset,” UCI Machine Learning Repository, 1999. [Online]. Available: <https://archive.ics.uci.edu/dataset/94/spambase>
3. Scikit-learn Developers, “Logistic Regression,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
4. Scikit-learn Developers, “Support Vector Machines,” scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/svm.html>
5. Scikit-learn Developers, “Tuning the hyper-parameters of an estimator,” scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/grid_search.html